

Roteiro de Video - AI Scribe Notes Management Tool

Duracao sugerida: 6 a 8 minutos.

Objetivo do video: mostrar que o projeto roda, explicar o fluxo principal e justificar as principais decisoes tecnicas sem entrar em detalhes desnecessarios.

Importante antes de gravar:

- Nao mostrar `OPENAI\_API\_KEY`, `AWS\_ACCESS\_KEY\_ID` ou `AWS\_SECRET\_ACCESS\_KEY`.
- Deixar Docker rodando antes ou iniciar a gravacao mostrando o comando.
- Deixar um audio curto pronto para upload.
- Deixar o AWS Console aberto no bucket S3.
- Criar uma nota de texto e uma nota de audio durante o video.

0:00 - 0:30 | Introducao

O que mostrar na tela

- Tela inicial do app em `http://localhost:5173`.
- Lista de pacientes/notas, se ja houver.

O que falar

Hi, this is the AI Scribe Notes Management Tool.

The goal of this project is to create and view AI-generated clinical notes associated with mock patient records.

The app supports typed clinical notes and audio uploads. Audio files are stored in AWS S3, the content is processed with OpenAI, and the resulting notes are persisted in PostgreSQL.

Ponto principal

Mostre logo que e um produto funcional, nao uma landing page.

0:30 - 1:20 | Setup e Como Rodar

O que mostrar na tela

- Terminal na raiz do projeto.
- Arquivo `docker-compose.yml`.

- Se quiser, mostre `.env` com as chaves escondidas.

Comandos para mostrar

```
docker compose up --build
```

Ou, se ja estiver rodando:

```
docker compose ps
```

O que falar

The project is fully Dockerized.

Docker Compose starts the React frontend, the Node.js and TypeScript backend, PostgreSQL, and the supporting services needed for local development.

The application is configured through environment variables, including the PostgreSQL connection string, the OpenAI API key, and the AWS S3 bucket configuration.

For this final setup, I configured a real AWS S3 bucket for audio storage and OpenAI for transcription and clinical note generation.

Ponto principal

Deixe claro que o avaliador consegue rodar com um unico comando.

1:20 - 2:40 | Demo 1: Criar Nota com Texto

O que mostrar na tela

- Frontend.
- Selecionar um paciente.
- Digitar uma nota clinica.
- Clicar em `Save note`.
- Abrir a nota na lista.

Texto para usar no formulario

```
Patient reports improved mobility after physical therapy. Mild soreness noted. No shortness of breath. Caregiver reviewed medication schedule.
```

O que falar

First, I will create a note using typed input.

I select an existing seeded patient, enter the clinical text, and submit the form.

Depois de salvar:

When the form is submitted, the backend receives the raw text and sends it to OpenAI to generate a concise SOAP-style clinical note.

The application stores both the raw input and the processed note in PostgreSQL.

Ao abrir a nota:

In the detail view, we can see the generated summary, the original raw input, and the related patient metadata displayed alongside the note.

Ponto principal

Mostre:

- Nome do paciente.
 - Preview na lista.
 - Summary.
 - Raw input.
 - Painel com dados do paciente.
-

2:40 - 4:10 | Demo 2: Criar Nota com Audio

O que mostrar na tela

- Selecionar paciente.
- Fazer upload de um arquivo de audio.
- Salvar.
- Abrir a nota criada.
- Mostrar bucket no AWS S3.

O que falar

Now I will create a note using audio input.

The backend accepts the uploaded audio file, stores it in AWS S3, sends the file to OpenAI for transcription, and then uses the transcription to generate the structured SOAP note.

Depois de salvar:

This gives us the full audio workflow: file upload, S3 persistence, transcription, note generation, and database persistence.

Ao mostrar o S3:

The audio file itself is not stored in the database.

PostgreSQL stores the note data and the S3 object reference, while the actual audio file lives in the S3 bucket under the `audio/` prefix.

Ponto principal

No AWS Console, mostrar:

```
S3 > ai-scribe-notes-douglas-wagner > audio/
```

Se o link do audio no app nao abrir publicamente, fale:

The app stores the S3 object reference. In a production setup, audio playback would usually use signed URLs or a controlled download endpoint instead of making the bucket public.

4:10 - 5:10 | Backend e API

O que mostrar na tela

- Estrutura do backend:

```
backend/src/routes  
backend/src/services  
backend/src/db
```

- Arquivos principais:

```
backend/src/routes/notes.ts  
backend/src/services/openaiService.ts  
backend/src/services/storageService.ts  
backend/src/services/noteService.ts
```

O que falar

The backend is built with Node.js, TypeScript, and Express.

I organized it around routes and services. Routes handle HTTP concerns, while services encapsulate business logic like note creation, OpenAI processing, patient lookup, and S3 storage.

The main API endpoints are `GET /api/patients`, `GET /api/notes`, `GET /api/notes/:id`, and `POST /api/notes`.

The note creation endpoint accepts multipart form data, so the same endpoint can support typed text, audio upload, or both.

Ponto principal

Fale de separacao de responsabilidades.

5:10 - 6:00 | Banco de Dados e Seed

O que mostrar na tela

- ``db/schema.sql`` ou ``backend/src/db/migrate.ts``.
- ``backend/src/db/seed.ts``.

O que falar

PostgreSQL is the source of truth for patients and notes.

On startup, the backend runs a small migration and seeds three mock patients if they do not already exist.

The ``notes`` table stores the raw text, the processed note, the input type, timestamps, and optional audio metadata such as the S3 storage key and audio URL.

Each note belongs to a patient through ``patient_id``, which allows the detail view to show the clinical note and patient context together.

Ponto principal

Mostre que o banco foi pensado para o fluxo pedido, sem exagero.

6:00 - 7:00 | Decisoões Tecnicas e Trade-offs

O que mostrar na tela

- Voltar para o app ou README.
- Opcional: mostrar a secao ``Assumptions and Shortcuts``.

O que falar

I focused on shipping a clean vertical slice rather than adding extra features.

Authentication, authorization, background jobs, and advanced clinical review workflows are intentionally out of scope for this take-home.

AI processing currently happens synchronously during note creation. This keeps the implementation simple and easy to demo.

In a production system, I would likely move transcription and summarization into a background job queue and expose processing status to the frontend.

I used S3 for audio storage because audio files should not live directly in the database. The database stores metadata and references, while S3 stores the binary files.

I also kept the frontend minimal and workflow-focused: create a note, list notes, and inspect the note with patient metadata.

Ponto principal

Mostre maturidade: voce sabe o que ficou fora e por que.

7:00 - 7:30 | Encerramento

O que mostrar na tela

- Tela de detalhe de uma nota criada.

O que falar

To summarize, this project implements the core workflow requested in the take-home: seeded patients, note creation from text or audio, OpenAI transcription and SOAP-style generation, AWS S3 audio storage, PostgreSQL persistence, and a React UI for listing and viewing notes with patient details.

Thank you for reviewing the project.

Checklist de Gravacao

Antes de comecar:

- `docker compose up --build` rodando.
- Frontend abre em `http://localhost:5173`.
- Backend responde em `http://localhost:4000/api/health`.
- OpenAI configurada no `.env`.
- AWS S3 configurado no `.env`.
- Bucket S3 aberto no console.
- Audio curto pronto para upload.
- Terminal e editor com tamanho de fonte legivel.
- Chaves escondidas.

Durante o video:

- Falar devagar.
- Mostrar o produto funcionando primeiro.
- Evitar explicar codigo linha por linha.
- Nao gastar tempo demais no Docker.
- Nao mostrar segredos.

Depois de gravar:

- Conferir se audio da gravacao esta claro.
- Conferir se o S3 aparece no video.

- Conferir se a criacao de nota texto e audio ficou visivel.
 - Conferir se nenhuma chave apareceu na tela.
-

Versao Curta Para Memorizar

This is a Dockerized AI Scribe Notes Management Tool built with React, Node.js, TypeScript, Express, PostgreSQL, AWS S3, and OpenAI.

It seeds mock patients, lets users create notes from text or audio, stores audio in S3, uses OpenAI for transcription and SOAP-style summarization, persists notes in PostgreSQL, and displays note details with related patient metadata.